

COURSE 3 - DATA ENGINEERING 2026

C3-070 - Streaming & Event-Driven Pipelines Beginner-First Lesson Book

Patient teacher route from event mental models to Kafka, schemas/CDC, late data, Structured Streaming, checkpoint recovery, lakehouse sinks and operations.

Artifact	C3-070
Status	2026 - LEARNER EDITION
Phase	C3-06 Streaming & Event-Driven Pipelines 13 lessons 34-46 focused hours
Prerequisite	C3-05 Gate PASS; C2B streaming/lakehouse concepts may be recalled but Kafka/Spark streaming implementation is taught here.
Current lane	Apache Kafka 4.3.1 Spark Structured Streaming 4.2.0 Delta Lake 4.4.0 Debezium 3.6.2.Final
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

House teaching rule: mental picture -> tiny event example -> guided Kafka/streaming action -> expected result -> why it worked -> break/fix -> independent changed case -> validation -> replay/state/production reasoning -> evidence. A running stream is not mastery if duplicates, late data, offsets, checkpoints or sink state are wrong.

This book is your teacher

Use this book as the primary teacher. External videos and official documentation are visual or current-behavior companions, never hidden prerequisites.

The locked beginner-first lesson contract

- | | |
|-----------------------------------|---|
| 01. Why it matters | 09. Why it worked |
| 02. What you learn | 10. Common beginner mistakes |
| 03. Prerequisite / starting state | 11. If stuck / recovery |
| 04. Picture it in your head | 12. Small independent task |
| 05. New words before use | 13. Validate |
| 06. Useful video/source | 14. Teach it back |
| 07. Follow with me once | 15. Protected review + changed retry |
| 08. Expected result | 16. Evidence/mastery + exact next route |

Rules that protect your learning

- Save a genuine attempt before opening a protected review.
- After review, close it and solve a changed retry - do not copy the reviewed answer.
- A command that runs is not mastery: validate grain, keys, counts, state, failure/recovery and evidence.
- If a new engineering concept appears, this book explains it from first principles even when Bridge skills are assumed.
- When stuck, repair only the named weak competency; do not restart the whole phase.

Contents / Index

Actual page numbers after the master-audit repair. Mentor routes use these same pages.

Front matter	Page
This book is your teacher	2
Contents / Index	3
Module map + practice/review/Gate route	4
Phase-specific study notes	5
Lessons	Start
C3-06-L01 Batch vs streaming and event-driven mental models	6
C3-06-L02 Kafka brokers, topics, partitions and replication	9
C3-06-L03 Producers, consumers and consumer groups	12
C3-06-L04 Offsets, retention and replay	15
C3-06-L05 Event serialization: JSON/Avro/Protobuf, schema registry and evolution	18
C3-06-L06 Kafka Connect and CDC integration patterns	21
C3-06-L07 Delivery semantics, duplicates and idempotent consumers	24
C3-06-L08 Event time vs processing time and late arrivals	27
C3-06-L09 Watermarks and windowed aggregations	30
C3-06-L10 Spark Structured Streaming DataFrames	33
C3-06-L11 Checkpointing, restart and state recovery	36
C3-06-L12 Streaming into Delta/lakehouse tables	39
C3-06-L13 Lag, operational failure modes and stream monitoring	42
Mastery checklist + revision quick reference	45

Module map + protected learning route

Streaming & Event-Driven Pipelines

- 01 Prerequisite / prior-phase Gate
- 02 Lesson mental model + vocabulary
- 03 Follow guided example once
- 04 Matching independent practice
- 05 Save genuine attempt
- 06 Protected review unlocks
- 07 Close review + changed retry
- 08 Validate + explain back
- 09 Master lesson + save evidence
- 10 Mini-Lab / integrated drill
- 11 Protected Gate A
- 12 If needed: protected review -> targeted repair -> changed micro-proof
- 13 Explicitly assign fresh Gate B/C only after repair evidence
- 14 Phase PASS -> next phase

What counts as mastery

- You can rebuild the competency without the lesson/review open.
- Your validation catches a plausible wrong result, not only syntax failure.
- You can state one failure mode and the recovery step.
- Your evidence names the input/output or artifact and the independent check used.

Revision rule

Revise from evidence: weak validation, weak recovery or weak explanation sends you back only to the matching lesson/repair route. Completion percentage never overrides a failed Gate.

How to use this book

Every lesson occupies exactly three pages so Mentor can open the correct page instead of dumping you at the front of a long PDF.

Study loop

- Page A: why -> outcome -> prerequisite -> mental picture -> vocabulary -> source/visual.
- Page B: tiny example -> follow once -> expected result -> why it worked -> deliberate break -> repair.
- Page C: common mistakes -> recovery ladder -> changed independent task -> validation -> production reasoning -> teach-back -> protected review -> evidence.

External videos are visual support, never a Gate dependency. If an old visual shows obsolete Kafka ZooKeeper setup, use it only for the durable topic/partition/producer/consumer mental model and follow the current Kafka 4.3.1 KRaft docs for commands.

C3-06-L01 - Batch vs streaming and event-driven mental models

Focused effort: 2.0 h | Matching practice: P01 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Choose batch, micro-batch or event-driven processing from latency, ordering, replay and state requirements rather than trendiness.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture two kitchens. One cooks a full day of orders at midnight; the other reacts as each ticket arrives. Streaming is not simply faster batch - the second kitchen must remember what it already served, handle duplicate tickets, late tickets and restarts while new work keeps arriving.

5. New words before we use them

Batch: bounded work processed as a set. Stream: unbounded event sequence. Event-driven: work triggered by events. Latency: delay from event to useful result. State: memory of prior events needed for correctness.

6. Useful source / visual

[Apache Kafka 4.3 - Introduction](#)

[freeCodeCamp - Apache Kafka for Beginners \(preserved R057\)](#)

Useful segment: Use the topics/partitions/producers/consumers sections; skip obsolete ZooKeeper-era details if shown.

C3-06-L01 - Follow with me once

7. Tiny worked example / prediction

Compare nightly settlement, 5-minute fraud features, parcel-status notifications and a one-time historical rebuild. State bounded/unbounded input and acceptable latency.

8. Follow with me once - real project action

Complete architecture/L01_mode_decisions.md for five supplied workloads.

9. Expected result

A decision matrix that selects batch, micro-batch or event-driven processing from latency, replay, state and operating-cost requirements - not fashion.

10. Why it worked

The business decision first defines required freshness and recovery behavior. Architecture then follows from a bounded/unbounded workload and operational burden instead of treating streaming as automatically superior.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If every workload becomes streaming, restate the business latency and replay requirement. A daily dashboard rarely needs second-level event processing.

C3-06-L01 - Now prove it yourself

13. Common beginner mistakes

- Choosing streaming because it sounds more advanced.
- Ignoring replay/state/late-data costs.
- Calling a five-minute scheduled batch "real time" without stating latency needs.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Change one workload from "alert within 10 seconds" to "report by 8 AM" and re-evaluate the architecture.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Production choice includes SLO, replay window, state size, team on-call capacity and cost. Streaming is justified when the business decision cannot wait for a simpler bounded run.

18. Teach it back

Explain batch vs streaming and event-driven mental models in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P01 evidence first. Then open C3-072 R01. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

mode decision matrix + changed decision. You can defend a fresh batch/streaming choice using latency, replay and operating cost.

C3-06-L02 - Kafka brokers, topics, partitions and replication

Focused effort: 3.0 h | Matching practice: P02 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Explain broker/topic/partition/replica, ordering scope, keyed partitioning and why local RF=1 is not production availability.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture a railway network. A topic is the route, partitions are parallel tracks, brokers are stations holding track segments, and replicas are backup copies. Ordering is guaranteed along one track, not across every track at once.

5. New words before we use them

Broker: Kafka server. Topic: named event log. Partition: ordered shard of a topic. Offset: position inside one partition. Replica: copy of a partition. Replication factor: number of copies.

6. Useful source / visual

[Apache Kafka 4.3 - Design](#)

[freeCodeCamp - Apache Kafka for Beginners \(preserved R057\)](#)

Useful segment: Use the broker/topic/partition/replication explanation; current Apache docs control KRaft-era behavior.

C3-06-L02 - Follow with me once

7. Tiny worked example / prediction

Sketch a 3-partition topic for order events keyed by order_id and place six events. Show offsets independently inside each partition.

8. Follow with me once - real project action

Complete architecture/L02_topic_design.md and the local command plan in kafka/KAFKA_4_3_1_LOCAL_LAB.md.

9. Expected result

A topic design with explicit partition key, per-partition ordering statement, and a clear note that a one-broker RF=1 laptop lab is not high availability.

10. Why it worked

Kafka keeps ordering and offsets per partition. Choosing a stable key colocates related events; replication protects data only when copies live on separate healthy brokers.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If you assume one global offset or ordering across partitions, redraw offsets per partition.

C3-06-L02 - Now prove it yourself

13. Common beginner mistakes

- Assuming total ordering across partitions.
- Using a random key when per-entity order matters.
- Calling RF=1 fault tolerant.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Switch the key from order_id to random UUID and explain the consequence for per-order ordering.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Partition count constrains parallelism and ordering. Replication, ISR health, retention and key distribution are operational design choices; the single-node lab teaches semantics only.

18. Teach it back

Explain kafka brokers, topics, partitions and replication in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P02 evidence first. Then open C3-072 R02. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

topic design + key transfer. You can design topic partitions/keys and explain the HA boundary.

C3-06-L03 - Producers, consumers and consumer groups

Focused effort: 3.0 h | Matching practice: P03 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Reason about producer keys, consumer-group assignment and scaling limits from partition count.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture a newspaper press. Producers print issues onto topic partitions. A consumer group is one delivery company whose drivers split neighborhoods; another delivery company can independently deliver the same newspaper without stealing copies from the first group.

5. New words before we use them

Producer: writes records. Consumer: reads records. Consumer group: consumers sharing work under one group id. Assignment: mapping partitions to consumers. Rebalance: reassignment when group membership/partitions change.

6. Useful source / visual

[Apache Kafka 4.3 - Consumer configs](#)

[freeCodeCamp - Apache Kafka for Beginners \(preserved R057\)](#)

Useful segment: Use the consumer-group/partition assignment portion; current group-protocol details come from Apache docs.

C3-06-L03 - Follow with me once

7. Tiny worked example / prediction

For three partitions, draw assignments with 1, 2, 3 and 5 consumers. Explain why another analytics group does not steal records from the first group.

8. Follow with me once - real project action

Complete architecture/L03_group_assignment.md and producer/consumer command notes.

9. Expected result

Correct partition-to-consumer assignments: one group shares partitions; a second group can independently read the same topic; extra consumers can be idle.

10. Why it worked

Group coordination divides partitions among members of one group. Group identity is what creates independent subscriptions, so another group gets its own positions.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If you say Kafka "sends each event to only one application," distinguish one consumer group from multiple independent groups.

C3-06-L03 - Now prove it yourself

13. Common beginner mistakes

- Thinking Kafka sends a record to only one application globally.
- Adding consumers beyond partition count and expecting more parallelism.
- Ignoring rebalances when scaling or restarting consumers.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Change the topic from 3 to 6 partitions and re-evaluate group parallelism and ordering.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Consumer scaling depends on partitions, processing time and rebalance behavior. Separate groups provide independent applications, not duplicate workers inside one group.

18. Teach it back

Explain producers, consumers and consumer groups in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P03 evidence first. Then open C3-072 R03. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

group assignment proof + transfer. You can predict assignment/scale from a fresh partition/group scenario.

C3-06-L04 - Offsets, retention and replay

Focused effort: 3.0 h | Matching practice: P04 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Treat offsets as per-partition read positions, distinguish committed progress from retention, and design replay without corrupting sinks.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture numbered pages in several notebooks. An offset is the next page position for one partition. Retention decides how long old pages remain. Replay means deliberately reading earlier pages again; it is safe only when downstream effects are designed for repetition.

5. New words before we use them

Committed offset: saved consumer progress. Retention: how long records remain available. Replay: rereading old offsets. auto.offset.reset: behavior when no valid committed offset exists. Lag: distance from consumer position to log end.

6. Useful source / visual

[Apache Kafka 4.3 - Consumer offset tracking](#)

Book + official documentation are sufficient; no external video is required for mastery.

C3-06-L04 - Follow with me once

7. Tiny worked example / prediction

Use the supplied simulator to process two partitions, save offsets, replay an earlier range, and compare source versus sink identity.

8. Follow with me once - real project action

Complete simulator P04 and evidence/L04_replay_contract.md.

9. Expected result

A replay plan that states starting position, retention risk, committed-offset behavior and how downstream state avoids duplicate business effects.

10. Why it worked

Kafka stores records independently of consumer position. That separation makes replay possible, while committed offsets and retention determine what can be resumed or reread.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If replay duplicates output, repair the sink identity/dedupe rule rather than disabling replay.

C3-06-L04 - Now prove it yourself

13. Common beginner mistakes

- Treating committed offset as record deletion.
- Replaying without idempotent sinks.
- Assuming earliest/latest is safe without retention and business requirements.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Reduce simulated retention below the lag window and explain the data-loss risk.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Replay is a product capability only if retention, offsets and downstream idempotence are designed together. A replay request without a bounded range and validation control is an incident risk.

18. Teach it back

Explain offsets, retention and replay in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P04 evidence first. Then open C3-072 R04. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

offset/checkpoint evidence + replay control. You can design a fresh replay plan with offset, retention and sink safety.

C3-06-L05 - Event serialization: JSON/Avro/Protobuf, schema registry and evolution

Focused effort: 2.5 h | Matching practice: P05 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Separate payload serialization from schema governance and classify compatible versus breaking event evolution.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture a shipping form that evolves. JSON, Avro and Protobuf are envelope formats; a schema registry is the rulebook that decides whether a new form version can still be understood by old/new readers. Compatibility is a contract, not just successful serialization.

5. New words before we use them

Serialization: encoding data to bytes. Schema: field/type contract. Avro/Protobuf/JSON Schema: common schema formats. Subject: schema-registry namespace. Compatibility: allowed relationship between old/new schemas. Evolution: changing schema over time.

6. Useful source / visual

[Confluent - Schema Evolution & Compatibility](#)

[Schema Registry visual \(preserved R058\)](#)

Useful segment: Watch for the mental model only; the book/current Confluent docs define compatibility modes.

C3-06-L05 - Follow with me once

7. Tiny worked example / prediction

Compare purchase-event v1 and v2 where v2 adds optional source_device. Classify rename, required-field add, type change and field removal risks.

8. Follow with me once - real project action

Complete contracts/event_v1.json, event_v2.json and contracts/L05_COMPATIBILITY.md.

9. Expected result

A schema-change decision that identifies whether a field change is compatible under the chosen policy and what producer/consumer rollout order is safe.

10. Why it worked

Schema compatibility is evaluated between versions under an explicit policy. Producer and consumer rollout becomes safer because breaking changes are detected before bytes reach every downstream service.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If you equate "valid JSON" with "compatible event," test a consumer expecting v1 against v2.

C3-06-L05 - Now prove it yourself

13. Common beginner mistakes

- Treating valid JSON as a schema contract.
- Changing a required field without checking compatibility.
- Equating Schema Registry with Kafka itself.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Design v3 that adds an optional campaign_id while retaining replay compatibility.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Schema governance needs owner, compatibility mode, rollout sequence and rollback plan. Compatibility settings are necessary controls but do not validate business meaning.

18. Teach it back

Explain event serialization: json/avro/protobuf, schema registry and evolution in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P05 evidence first. Then open C3-072 R05. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

versioned contract + compatibility table. You can defend a fresh schema evolution against replayed older events.

C3-06-L06 - Kafka Connect and CDC integration patterns

Focused effort: 3.0 h | Matching practice: P06 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Explain source/sink connectors, CDC change envelopes, snapshot-to-stream transition and when Connect/Debezium is better than custom polling.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture a conveyor between a database and Kafka. Kafka Connect moves records using connector plugins; Debezium observes database change logs and emits change events. The connector is plumbing, but you still own source semantics, keys, deletes, schema evolution and recovery.

5. New words before we use them

Kafka Connect: connector runtime/API. Source connector: imports into Kafka. Sink connector: exports from Kafka. CDC: change data capture. Debezium: CDC connector platform. Tombstone/delete event: record signaling deletion semantics.

6. Useful source / visual

[Debezium 3.6 Documentation](#)

Book + official documentation are sufficient; no external video is required for mastery.

C3-06-L06 - Follow with me once

7. Tiny worked example / prediction

Map database -> Debezium source connector -> Kafka topic -> consumer/lakehouse sink. Distinguish row state from change event.

8. Follow with me once - real project action

Complete architecture/L06_cdc_flow.md using the provided synthetic CDC envelope examples.

9. Expected result

A CDC flow diagram showing database log -> Debezium/source connector -> Kafka topic -> consumer/sink, with key/delete/schema/restart responsibilities explicit.

10. Why it worked

CDC reads the database change log instead of repeatedly scanning whole tables. Kafka Connect standardizes movement, but business meaning still depends on keys, operation type and schema/change semantics.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If you flatten CDC to "latest row only," identify what delete/history/replay information was lost.

C3-06-L06 - Now prove it yourself

13. Common beginner mistakes

- Assuming CDC means "all rows every minute".
- Ignoring delete/tombstone semantics.
- Committing connector credentials/config secrets to Git.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Change one source update into delete + recreate and explain downstream semantics.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

CDC introduces operational dependencies on source logs, connector offsets, snapshot behavior, delete semantics and database permissions. Monitor connector health and source retention headroom.

18. Teach it back

Explain kafka connect and cdc integration patterns in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P06 evidence first. Then open C3-072 R06. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

CDC flow + envelope reasoning. You can interpret fresh CDC envelopes and defend connector versus custom-ingestion boundaries.

C3-06-L07 - Delivery semantics, duplicates and idempotent consumers

Focused effort: 3.0 h | Matching practice: P07 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Reason about at-most-once/at-least-once/exactly-once boundaries and build idempotent consumption around stable event identity.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture a courier who may ring twice after a network timeout. At-least-once delivery means duplicates can happen. Idempotence means ringing twice does not create two business side effects. Exactly-once claims are always scoped to particular components and transactions.

5. New words before we use them

At-most-once: may lose, avoids duplicates. At-least-once: retries can duplicate. Idempotent consumer: repeated event causes same final business state. Event key: stable identity/routing key. Deduplication: detecting repeated events.

6. Useful source / visual

[Apache Kafka 4.3 - Consumer configs](#)

Book + official documentation are sufficient; no external video is required for mastery.

C3-06-L07 - Follow with me once

7. Tiny worked example / prediction

Process a batch with an exact duplicate event_id and two legitimate lifecycle events for the same order_id.

8. Follow with me once - real project action

Complete src/stream_core.py TODO-free reference functions by studying tests, then implement your own consumer_TODO.py with event_id idempotency.

9. Expected result

Reprocessing the same event_id does not create duplicate accepted business state; retry behavior and the exact guarantee boundary are documented.

10. Why it worked

Retries are normal in distributed systems. Stable event identity plus idempotent business writes makes repeated delivery converge to one correct state.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If you dedupe on order_id, restore legitimate multiple lifecycle events.

C3-06-L07 - Now prove it yourself

13. Common beginner mistakes

- Claiming exactly-once end to end without defining scope.
- Deduping on an unstable/generated arrival id.
- Committing progress before a non-idempotent side effect is safe.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Change the domain to payment attempts where repeated order_id can be legitimate and choose a new idempotency key.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Delivery guarantee claims must name source, processor and sink boundaries. Prefer evidence of convergence under duplicate/retry/restart over marketing labels.

18. Teach it back

Explain delivery semantics, duplicates and idempotent consumers in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P07 evidence first. Then open C3-072 R07. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

dedupe/replay tests + key defense. You can design duplicate-safe consumption without deleting valid repeated events.

C3-06-L08 - Event time vs processing time and late arrivals

Focused effort: 3.0 h | Matching practice: P08 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Use event time for business-time windows, distinguish processing delay, and define a lateness policy from business impact.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture a parcel created at 09:05 but scanned into the system at 09:17. Event time answers when the business event happened; processing time answers when the system handled it. Late arrival is the gap between those clocks.

5. New words before we use them

Event time: time at source/business event. Processing time: time handled by processor. Ingest time: time received by platform. Late event: event arriving after newer event-time data. Clock skew: disagreement between clocks.

6. Useful source / visual

[Spark 4.2 - Structured Streaming DataFrame/Dataset APIs](#)

[Databricks Structured Streaming design visual \(preserved R059\)](#)

Useful segment: Use the event-time/late-data concepts; current Spark docs control exact API behavior.

C3-06-L08 - Follow with me once

7. Tiny worked example / prediction

Plot five events by event time and ingest time, including one 12-minute-late event.

8. Follow with me once - real project action

Run scripts/run_stream_simulator.py on batch 1 then batch 2 and inspect event-time versus arrival order.

9. Expected result

A timeline that correctly separates event time, ingest time and processing time and labels which events are late without rewriting source timestamps.

10. Why it worked

Business-time windows must follow event time, not machine arrival time. Separating clocks lets the system reason explicitly about lateness rather than pretending arrival order equals event order.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If you classify late as invalid automatically, separate "late but valid" from malformed/impossible timestamp.

C3-06-L08 - Now prove it yourself

13. Common beginner mistakes

- Using processing time for business-time windows.
- Overwriting source event time with ingest time.
- Calling every out-of-order event bad data.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Change allowed lateness from 10 to 3 minutes and identify which business metric changes.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Event-time correctness depends on source clock quality and timezone policy. Late-event handling should be explicit: update, quarantine, compensating correction or business cutoff.

18. Teach it back

Explain event time vs processing time and late arrivals in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P08 evidence first. Then open C3-072 R08. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

timeline + changed lateness analysis. You can defend event-time and lateness from a fresh timeline.

C3-06-L09 - Watermarks and windowed aggregations

Focused effort: 3.0 h | Matching practice: P09 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Build event-time windows, explain watermark as state/lateness control, and validate late-event consequences without oversimplifying it.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture a closing gate that moves forward behind the newest event. A watermark is the system saying, "I will still tolerate events this late." A time window groups events by event-time buckets; state must be retained until the system can safely finalize or update those buckets.

5. New words before we use them

Window: event-time bucket. Watermark: threshold used to bound tolerated lateness/state. Stateful aggregation: result requiring prior events. Output mode: how changing results are emitted. State store: maintained aggregation/join state.

6. Useful source / visual

[Spark 4.2 - Event-time windows and watermarking](#)

[Databricks Structured Streaming design visual \(preserved R059\)](#)

Useful segment: Focus on event-time windows/watermark mental model; no timestamp is invented.

C3-06-L09 - Follow with me once

7. Tiny worked example / prediction

Create 5-minute paid-event windows with a 10-minute lateness policy. Compare on-time, late-but-accepted and too-late events in the simulator.

8. Follow with me once - real project action

Complete evidence/L09_watermark_experiment.md and the Spark TODO plan using withWatermark.

9. Expected result

A window/watermark experiment where the predicted accepted/late/state behavior matches the simulator or live Structured Streaming evidence.

10. Why it worked

Watermarks provide a bounded lateness/state policy. Windows define grouping boundaries; together they control when state can be updated or eventually discarded.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If a late event surprises you, record the prior max event time, watermark and target window instead of guessing.

C3-06-L09 - Now prove it yourself

13. Common beginner mistakes

- Thinking watermark means "drop anything older than now minus X" without query context.
- Using windows without declaring timezone/event-time field.
- Keeping unbounded state because no lateness policy was chosen.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Repeat with 15-minute windows and a 5-minute watermark.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Watermark/window choices trade correctness delay, memory/state growth and lateness tolerance. Tune from observed arrival patterns and SLOs, not arbitrary round numbers.

18. Teach it back

Explain watermarks and windowed aggregations in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P09 evidence first. Then open C3-072 R09. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

window controls + fresh experiment. You can predict a changed window/watermark experiment and validate state/aggregate effects.

C3-06-L10 - Spark Structured Streaming DataFrames

Focused effort: 3.5 h | Matching practice: P10 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Build a typed file-based Structured Streaming query with normal DataFrame transformations and deterministic AvailableNow lab execution.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture an ordinary Spark DataFrame table whose rows never stop arriving. Structured Streaming lets you describe transformations with DataFrame/SQL APIs while Spark repeatedly executes incremental work and records query progress.

5. New words before we use them

Streaming DataFrame: unbounded DataFrame query. Trigger: when micro-batches/progress occur. StreamingQuery: running query handle. Query progress: observed rates/batch/state metrics. Kafka source: Structured Streaming connector to Kafka.

6. Useful source / visual

[Spark 4.2 - Structured Streaming + Kafka integration](#)

[Databricks Structured Streaming design visual \(preserved R059\)](#)

Useful segment: Focus on streaming DataFrame source/sink/checkpoint shape.

C3-06-L10 - Follow with me once

7. Tiny worked example / prediction

Read JSON files with explicit schema, parse timestamps, dedupe, watermark/window and write a checkpointed sink.

8. Follow with me once - real project action

Complete `spark/structured_stream_TODO.py`. Runtime evidence must include actual query progress only when Spark is installed.

9. Expected result

A Structured Streaming query definition that uses DataFrame APIs, has an explicit schema/source/sink/checkpoint, and produces inspectable query progress when run.

10. Why it worked

Structured Streaming reuses Spark SQL/DataFrame semantics while managing continuous execution and progress around them. The logical transformation stays declarative while the engine tracks streaming progress.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If you use `inferSchema` on an unbounded JSON stream, move schema to code/contract. If output differs after rerun, inspect checkpoint identity.

C3-06-L10 - Now prove it yourself

13. Common beginner mistakes

- Using legacy DStreams for a new Structured Streaming lesson.
- Forgetting checkpoint location for recoverable queries.
- Using collect() on an unbounded stream.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Point the same transformation at a second inbox with a fresh checkpoint and compare results.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

A production streaming job needs source/sink contracts, checkpoint storage, schema evolution policy, query progress monitoring, restart behavior and deployment ownership.

18. Teach it back

Explain spark structured streaming dataframes in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P10 evidence first. Then open C3-072 R10. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

Spark code + real progress evidence when available. You can assemble a fresh typed streaming DataFrame pipeline and explain trigger/checkpoint boundaries.

C3-06-L11 - Checkpointing, restart and state recovery

Focused effort: 3.0 h | Matching practice: P11 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Treat checkpoint as query identity/progress/state, prove same-checkpoint restart behavior, and reject unsafe checkpoint surgery.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture a bookmark plus working notes. A streaming checkpoint remembers source progress, commits and state needed to resume the same query. Deleting pieces of the bookmark folder is not a repair strategy; it can create a different query identity or inconsistent state.

5. New words before we use them

Checkpoint: persisted streaming query progress/state. Offset log: source progress records. Commit log: completed batch records. State store: persisted operator state. Restart: resume same query from checkpoint. New query: fresh identity/checkpoint.

6. Useful source / visual

[Spark 4.2 - Structured Streaming state/checkpoint notes](#)

[Databricks - Dive into Streaming Checkpoints & Best Practices \(preserved R060\)](#)

Useful segment: Checkpoint/restart examples; current Spark docs remain authority.

C3-06-L11 - Follow with me once

7. Tiny worked example / prediction

Run the pure simulator twice with the same checkpoint, then copy data to a fresh state path and compare replay behavior.

8. Follow with me once - real project action

Complete evidence/L11_restart_recovery.md and spark/checkpoint_contract.md.

9. Expected result

A same-checkpoint restart resumes progress/state; a fresh checkpoint behaves as a new query/replay. No manual deletion of random checkpoint internals.

10. Why it worked

A checkpoint is part of query identity, not scratch output. Reusing the same complete checkpoint lets the engine recover known progress/state; a new checkpoint intentionally starts a new lineage.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If a restart seems wrong, do not delete random checkpoint files. Restore the full checkpoint or intentionally start a new query and treat it as replay.

C3-06-L11 - Now prove it yourself

13. Common beginner mistakes

- Deleting random checkpoint files to make errors disappear.
- Changing stateful configuration under an old checkpoint without compatibility reasoning.
- Assuming startingOffsets overrides an existing checkpoint.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Change stateful window keys and explain why a new checkpoint/migration plan is safer.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Checkpoint compatibility is part of deployment/change management. Stateful query changes may require migration, replay or a new checkpoint; treat checkpoints as durable operational assets.

18. Teach it back

Explain checkpointing, restart and state recovery in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P11 evidence first. Then open C3-072 R11. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

restart proof + state-change decision. You can diagnose fresh checkpoint/restart scenarios without destructive trial-and-error.

C3-06-L12 - Streaming into Delta/lakehouse tables

Focused effort: 3.0 h | Matching practice: P12 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Design replayable Bronze/Silver streaming sinks using Delta, checkpointing and idempotent foreachBatch/upsert boundaries.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture two lakehouse floors. Bronze preserves source event evidence; Silver applies validated schema/dedupe/business state. Each streaming query needs its own checkpoint and sink contract so replay does not silently duplicate or overwrite the wrong layer.

5. New words before we use them

Bronze: evidence-preserving raw layer. Silver: validated/conformed state. Transactional sink: commits atomically. foreachBatch: per-micro-batch custom sink hook. MERGE/upsert: key-based update/insert. Separate checkpoint: independent query identity.

6. Useful source / visual

[Delta Lake - Table streaming reads and writes](#)

[Delta Lake 4.4.0 release context](#)

Useful segment: Use for Spark 4.2/Delta 4.4 compatibility and modern table-streaming context.

C3-06-L12 - Follow with me once

7. Tiny worked example / prediction

Design Kafka -> Bronze Delta append -> Silver dedupe/upsert with separate checkpoints and recovery rules.

8. Follow with me once - real project action

Complete lakehouse/L12_STREAMING_TABLE_CONTRACT.md and delta/foreach_batch_TODO.py.

9. Expected result

A Bronze/Silver sink design with separate checkpoints, deterministic event keys, replay rules and transactional/idempotent boundaries.

10. Why it worked

Delta/lakehouse transactions protect table commits, but custom foreachBatch/MERGE logic still needs stable keys and deterministic behavior. Separate query checkpoints prevent progress/state collisions.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If Bronze and Silver share one checkpoint or overwrite raw event history, separate query identity and evidence-preserving storage.

C3-06-L12 - Now prove it yourself

13. Common beginner mistakes

- Sharing one checkpoint between Bronze and Silver queries.
- Using append-only custom sinks where replay can duplicate state.
- Overwriting raw evidence needed for backfill/recovery.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Add a replay/backfill path for a historical offset range without corrupting live progress.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Use evidence-preserving Bronze and validated Silver to separate ingestion fidelity from business state. Replay/backfill paths should not silently share live progress or duplicate commits.

18. Teach it back

Explain streaming into delta/lakehouse tables in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P12 evidence first. Then open C3-072 R12. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

Bronze/Silver contracts + replay plan. You can defend a fresh lakehouse sink/replay design and its idempotency boundary.

C3-06-L13 - Lag, operational failure modes and stream monitoring

Focused effort: 3.0 h | Matching practice: P13 | Prerequisite: prior C3-06 lesson mastered/remediated; C3-05 Gate PASS is assumed.

1. Why it matters

This competency is part of production stream correctness: replay, state and failure behavior matter as much as a successful happy-path output.

2. What you learn

Diagnose backlog, stalled source, slow sink, state growth and retention risk using source/processing/state metrics plus a recovery runbook.

3. Prerequisite / what you already know

You already know Python, SQL, ingestion, lakehouse fundamentals and Spark batch concepts. This lesson teaches the new streaming/event behavior from first principles; live Kafka/Structured Streaming operation is never assumed.

4. Picture it in your head

Picture an operations dashboard with three gauges: how fast events arrive, how fast they are processed, and how much backlog/state is growing. "RUNNING" is only a process status; health comes from lag, throughput, batch duration, state, watermark and actionable runbooks.

5. New words before we use them

Consumer lag: records behind log end. Input rate: records arriving. Processing rate: records handled. Batch duration: processing time per trigger. Backlog: unprocessed work. SLO: measurable service objective. Runbook: operator response procedure.

6. Useful source / visual

[Apache Kafka 4.3 - Basic operations / consumer lag](#)

[Spark 4.2 Structured Streaming guide](#)

Useful segment: Use query progress/state metrics and consumer-group lag evidence.

C3-06-L13 - Follow with me once

7. Tiny worked example / prediction

Given three incident snapshots, distinguish source stall, processing backlog and state-growth problem.

8. Follow with me once - real project action

Complete monitoring/L13_OPERATOR_DASHBOARD.md and runbooks/STREAM_RECOVERY_TODO.md.

9. Expected result

An operator view that distinguishes source stall, processing backlog and state growth, with thresholds tied to retention/SLO risk and a concrete recovery action.

10. Why it worked

Lag/rates/state describe different failure classes. Tying alerts to a runbook turns metrics into an operating system for the pipeline instead of decorative dashboards.

11. Break it on purpose

Change one streaming assumption related to this lesson - key, partition, offset, schema version, duplicate, event time, lateness, checkpoint or sink rule. Predict the failure first, run/inspect only the required scope, then keep broken evidence beside repaired evidence.

12. Repair from evidence

If you alert on "lag > 0", tie thresholds to normal traffic, recovery time and retention headroom.

C3-06-L13 - Now prove it yourself

13. Common beginner mistakes

- Alerting on any non-zero lag.
- Treating RUNNING as healthy.
- Recording metrics without owner/action/retention headroom.

14. Stuck? recovery ladder

1) State event identity and business time. 2) Identify source progress/offset/checkpoint boundary. 3) Restate intended sink state and replay rule. 4) Reproduce the smallest failing event set. 5) Compare expected vs observed identity/count/time/state. 6) Reveal only the next command/API shape. 7) Open protected review only after a saved genuine attempt.

15. Small independent changed task

Change source traffic 5x while sink throughput stays fixed; calculate qualitative backlog direction and escalation action.

16. Validate

Compare observed events/keys/windows/state/offset or checkpoint evidence against an independent control. Explain any intentional duplicates, late events or rejected records instead of hiding them.

17. Production reasoning

Monitor source health, lag/backlog, processing rate, batch duration, state size, watermark and sink errors together. Every alert needs an owner, severity and validated recovery step.

18. Teach it back

Explain lag, operational failure modes and stream monitoring in 60-90 seconds using the project as evidence. Include one failure/replay case.

19. Protected review + changed retry

Save P13 evidence first. Then open C3-072 R13. Close review and solve a changed case with at least one different event/key/time/state rule. Never mark mastery from reading the review.

20. Evidence / mastery

dashboard spec + recovery runbook. You can triage a fresh stream incident and state detect -> diagnose -> recover -> validate.

Mastery checklist + revision quick reference

- ☐ Every lesson: guided rebuild completed, genuine independent attempt saved, validation recorded.
- ☐ Protected review opened only after the attempt; changed retry solved after closing the review.
- ☐ Explain-back: business/engineering purpose, grain or state, one failure mode, one recovery step, one validation check.
- ☐ Mini-Lab / integrated drill completed with reproducible evidence.
- ☐ Gate A attempted independently. If NEEDS_REPAIR: review viewed, targeted repair completed, changed micro-proof recorded, then a fresh B/C assigned explicitly.
- ☐ No unresolved critical failure, secret/private data leakage, fabricated live-run evidence or copied Gate solution.
- ☐ Evidence is saved with enough context for another person to reproduce or inspect it.

Revision quick reference

Wrong result / wrong grain	Return to validation + data contract / grain lesson; reproduce smallest failing case.
Works once, rerun breaks	Return to idempotence/state/checkpoint/recovery lesson; prove clean replay.
Cannot explain failure	Return to mental model + break/fix section; write the failure boundary in your own words.
Gate NEEDS_REPAIR	Open only its protected review, repair named weak competency, solve changed micro-proof, then assign fresh retest.
Tool/vendor changed	Use the current official source for syntax/UI, but keep the book's durable engineering contract and validation.